```
# Diagnóstico: quantas linhas o arquivo REALMENTE tem?
import subprocess

ARQUIVO = "dados_eleitorais_RN_2T.csv"   # ajuste o nome

# Conta linhas brutas (rápido, não carrega na memória)
print("Contando linhas do arquivo original...")
resultado = subprocess.run(["wc", "-l", ARQUIVO], capture_output=True, text=True)
print(f"Linhas brutas (wc -l): {resultado.stdout.strip()}")

# Conta registros reais com o parser do pandas
print("\nContando registros com o parser do pandas...")
total = 0
for bloco in pd.read_csv(ARQUIVO, sep=";", encoding="latin-1",
                         dtype=str, keep_default_na=False,
                         chunksize=200_000, low_memory=False):
    total += len(bloco)
print(f"Registros lidos pelo pandas: {total:,}")

# Conta o arquivo gerado
print("\nContando o arquivo enxuto...")
total_enxuto = 0
for bloco in pd.read_csv("dados_eleitorais_enxuto.csv", sep=";",
                         encoding="utf-8-sig", dtype=str,
                         keep_default_na=False, chunksize=200_000):
    total_enxuto += len(bloco)
print(f"Registros no enxuto: {total_enxuto:,}")
```

```
Contando linhas do arquivo original...
Linhas brutas (wc -l): 63677 dados_eleitorais_RN_2T.csv

Contando registros com o parser do pandas...
Registros lidos pelo pandas: 63,676

Contando o arquivo enxuto...
Registros no enxuto: 63,676
```

```
import pandas as pd
import os

ARQUIVO_ORIGINAL = "dados_eleitorais_RN_2T.csv"
ARQUIVO_SAIDA    = "dados_eleitorais_enxuto.csv"
ENCODING         = "latin-1"
SEPARADOR        = ";"
CHUNK            = 500_000

COLUNAS_REMOVER = [
    "DT_EMISSAO_BU", "DT_BU_RECEBIDO",
    "DT_CARGA_URNA_EFETIVADA", "CD_TIPO_URNA", "DS_TIPO_URNA",
    "NR_URNA_EFETIVADA", "CD_CARGA_1_URNA_EFETIVADA",
    "CD_CARGA_2_URNA_EFETIVADA", "CD_FLASHCARD_URNA_EFETIVADA",
]

primeiro = True
total_linhas = 0

for bloco in pd.read_csv(
    ARQUIVO_ORIGINAL,
    sep=SEPARADOR,
    encoding=ENCODING,
    dtype=str,
    keep_default_na=False,
    chunksize=CHUNK,
    low_memory=False,
):
    # Valida só no primeiro bloco
    if primeiro:
        faltando = [c for c in COLUNAS_REMOVER if c not in bloco.columns]
        if faltando:
            raise ValueError(f"Colunas não encontradas: {faltando}")
        print(f"Colunas no original: {len(bloco.columns)}")

    bloco = bloco.drop(columns=COLUNAS_REMOVER)

    bloco.to_csv(
        ARQUIVO_SAIDA,
        sep=";",
        encoding="utf-8-sig",
        index=False,
        mode="w" if primeiro else "a",
```

```
            header=primeiro,
        )

        total_linhas += len(bloco)
        print(f"Processadas {total_linhas:,} linhas...")
        primeiro = False

print(f"\nTotal: {total_linhas:,} linhas")
print(f"Arquivo: {ARQUIVO_SAIDA} ({os.path.getsize(ARQUIVO_SAIDA)/1024/1024:.2f} MB)")
```

```
Colunas no original: 45
Processadas 63,676 linhas...

Total: 63,676 linhas
Arquivo: dados_eleitorais_enxuto.csv (16.72 MB)
```

Célula 1 — Preparação do ambiente

🧰 Preparação: confere o Ollama, o servidor e o modelo

```
# @title 🧰 Preparação: confere o Ollama, o servidor e o modelo
import os, shutil, subprocess, sys, time, requests

OLLAMA_URL = "http://localhost:11434"
MODEL = "qwen2.5:3b"
IN_COLAB = "google.colab" in sys.modules
sh = get_ipython().system

def server_is_up() -> bool:
    try:
        return requests.get(f"{OLLAMA_URL}/api/version", timeout=2).ok
    except requests.RequestException:
        return False

# 1 · O programa ollama existe?
if shutil.which("ollama"):
    print("✅ Ollama já instalado")
elif IN_COLAB:
    print("⏳ instalando o Ollama (cerca de 1 minuto)...")
    sh("apt-get install -y -qq zstd > /dev/null 2>&1 || "
       "(apt-get update -qq > /dev/null 2>&1 && apt-get install -y -qq zstd > /dev/null 2>&1)")
    sh("curl -fsSL https://ollama.com/install.sh | sh 2>&1 | tail -n 2")
    if not shutil.which("ollama"):
        raise RuntimeError("a instalação falhou. Rode a célula de novo.")
    print("✅ Ollama instalado")
else:
    raise RuntimeError("Ollama não encontrado. Instale em https://ollama.com/download")

# 2 · O servidor está de pé?
if server_is_up():
    print("✅ servidor já está de pé")
else:
    print("⏳ subindo o servidor...")
    subprocess.Popen(["ollama", "serve"], stdout=open("ollama.log", "w"),
                     stderr=subprocess.STDOUT, start_new_session=True)
    for _ in range(30):
        if server_is_up():
            break
        time.sleep(1)
    if not server_is_up():
        raise RuntimeError("o servidor não respondeu. Veja ollama.log")
    print("✅ servidor de pé")

# 3 · O modelo já foi baixado?
installed = [m["name"] for m in requests.get(f"{OLLAMA_URL}/api/tags").json()["models"]]
if MODEL in installed:
    print(f"✅ {MODEL} já baixado")
else:
    print(f"⏳ baixando {MODEL} (~2 GB)...")
    sh(f"ollama pull {MODEL}")
    print(f"✅ {MODEL} baixado")

version = requests.get(f"{OLLAMA_URL}/api/version").json()["version"]
print(f"\n🚀 pronto · Ollama {version} · modelo {MODEL}")
```

```
✅ Ollama já instalado
✅ servidor já está de pé
✅ qwen2.5:3b já baixado

🚀 pronto · Ollama 0.40.14 · modelo qwen2.5:3b
```

```
import requests, subprocess

# 1. GPU visível no runtime
print(subprocess.run(["nvidia-smi", "--query-gpu=name,memory.total,memory.used",
                      "--format=csv,noheader"], capture_output=True, text=True).stdout.strip())

# 2. O Ollama está usando CUDA?
log = subprocess.run(["grep", "-E", "library=CUDA|library=cpu", "ollama.log"],
                     capture_output=True, text=True).stdout
print("CUDA detectado" if "library=CUDA" in log else "⚠️ CPU — GPU não está sendo usada")

# 3. Teste real de velocidade
r = requests.post("http://localhost:11434/api/generate", json={
    "model": MODEL, "prompt": "Conte de 1 a 50.", "stream": False,
    "options": {"num_predict": 100}}).json()
print(f"{r['eval_count'] / (r['eval_duration'] / 1e9):.1f} tokens/s")
```

```
Tesla T4, 15360 MiB, 2201 MiB
CUDA detectado
71.3 tokens/s
```

Célula 2 — Utilitários

🔧 Utilitários: chamada ao modelo e impressão da conversa

```
# @title 🔧 Utilitários: chamada ao modelo e impressão da conversa
import json, re
import pandas as pd

def llm(messages, tools=None):
    """Uma chamada ao /api/chat do Ollama. Devolve o JSON inteiro."""
    body = {"model": MODEL, "messages": messages, "stream": False,
            "options": {"temperature": 0, "num_predict": 600}}
    if tools:
        body["tools"] = tools
    r = requests.post(f"{OLLAMA_URL}/api/chat", json=body, timeout=300)
    r.raise_for_status()
    return r.json()

def show(obj, limit=1600):
    """Imprime um dicionário como JSON legível, cortado se for longo."""
    text = json.dumps(obj, ensure_ascii=False, indent=2)
    print(text if len(text) <= limit else text[:limit] + "\n ... (cortado)")

ICON = {"system": "⚙️ ", "user": "👤", "assistant": "🤖", "tool": "🔧"}

def show_messages(messages):
    """A conversa inteira, uma linha por mensagem."""
    for i, m in enumerate(messages):
        if m.get("tool_calls"):
            what = "pede " + " e ".join(
                f'{c["function"]["name"]}({json.dumps(c["function"]["arguments"], ensure_ascii=False)})'
                for c in m["tool_calls"])
        else:
            what = (m.get("content") or "").replace("\n", " ")
        print(f"[{i}] {ICON[m['role']]} {m['role']:<9} | {what[:120]}{'…' if len(what) > 120 else ''}")

print("utilitários prontos")
```

```
utilitários prontos
```

Célula 3 — Parte 1: o gabarito (antes de qualquer LLM)

📊 Parte 1 · O gabarito: respondemos à mão, com pandas

```
# @title 📊 Parte 1 · O gabarito: respondemos à mão, com pandas
import os

CSV = "dados_eleitorais_enxuto.csv"
if not os.path.exists(CSV):
    from google.colab import files
    print("Suba o arquivo dados_eleitorais_enxuto.csv")
    files.upload()

# dtype=str preserva zeros à esquerda em NR_ZONA, NR_SECAO, CD_MUNICIPIO
```

```
bu = pd.read_csv(CSV, sep=";", encoding="utf-8-sig", dtype=str, keep_default_na=False)

# padroniza os nomes das colunas para o que as funções esperam
bu = bu.rename(columns={"NR_ZONA": "ZONA_N", "NR_SECAO": "SECAO_N"})

print(f"{len(bu)} linhas · {len(bu.columns)} colunas\n")
print("Colunas:", ", ".join(bu.columns))

# --- GABARITO 1: total de votos por cargo (individuais + agregadas)
bu["QT_VOTOS_N"] = pd.to_numeric(bu["QT_VOTOS"], errors="coerce").fillna(0)
bu["AGREGADA"] = bu["DS_SECOES_AGREGADAS"] != "#NULO#"

# Regra: individuais + agregadas, SEMPRE. Não há sobreposição.
# O risco é o oposto — filtrar agregadas remove votos reais.

# Total por cargo (individuais + agregadas)
total_por_cargo = (bu.groupby("DS_CARGO_PERGUNTA")["QT_VOTOS_N"]
                     .sum().reset_index()
                     .sort_values("QT_VOTOS_N", ascending=False))
print("=== Total de votos por cargo (individuais + agregadas) ===")
print(total_por_cargo.to_string(index=False))

# Ranking de seções — só individuais, porque agregadas não têm seção única
individuais = bu[~bu["AGREGADA"]]
gabarito_secoes = (individuais.groupby(["ZONA_N", "SECAO_N", "DS_TIPO_VOTAVEL"])["QT_VOTOS_N"]
                              .sum().reset_index()
                              .sort_values("QT_VOTOS_N", ascending=False))
print("\n=== Top 10 seções individuais ===")
print(gabarito_secoes.head(10).to_string(index=False))

# Verificação: agregadas por cargo
print("\n=== Agregadas por cargo ===")
print(bu[bu["AGREGADA"]].groupby("DS_CARGO_PERGUNTA")
        .agg(linhas=("QT_VOTOS_N", "size"), votos=("QT_VOTOS_N", "sum"))
        .to_string())

# --- GABARITO 2: seções com inconsistência de comparecimento ---

aptos = pd.to_numeric(bu["QT_APTOS"], errors="coerce")
comp  = pd.to_numeric(bu["QT_COMPARECIMENTO"], errors="coerce")
abst  = pd.to_numeric(bu["QT_ABSTENCOES"], errors="coerce")

check = bu[["ZONA_N", "SECAO_N", "QT_APTOS", "QT_COMPARECIMENTO", "QT_ABSTENCOES"]].copy()
check["SOMA"] = comp + abst
check["DIFERENCA"] = aptos - check["SOMA"]
inconsistentes = check[(check["DIFERENCA"].notna()) & (check["DIFERENCA"] != 0)]

print(f"\n=== GABARITO 2: seções com inconsistência ===")
print(f"{len(inconsistentes)} seções com QT_APTOS ≠ QT_COMPARECIMENTO + QT_ABSTENCOES")
if len(inconsistentes):
    print(inconsistentes.to_string(index=False))

# --- GABARITO 3: seções agregadas ---
agregadas = bu[bu["DS_SECOES_AGREGADAS"] != "#NULO#"]
print(f"\n=== GABARITO 3: seções agregadas ===")
print(f"{len(agregadas)} linhas com DS_SECOES_AGREGADAS preenchido")

# --- GABARITO 4: total geral e conferência por tipo de voto ---
print("\n=== GABARITO 4: total geral ===")
print(f"Total de votos no arquivo: {bu['QT_VOTOS_N'].sum():,.0f}")

por_tipo = (bu.groupby("DS_TIPO_VOTAVEL")["QT_VOTOS_N"]
              .agg(linhas="size", votos="sum").reset_index()
              .sort_values("votos", ascending=False))
print("\n=== Votos por tipo ===")
print(por_tipo.to_string(index=False))

# Conferência: votos de UM cargo vs comparecimento
print("\n=== Conferência: votos por cargo vs comparecimento ===")
for cargo in ["Governador", "Presidente"]:
    sub = individuais[individuais.DS_CARGO_PERGUNTA == cargo]

    votos = (sub.groupby(["ZONA_N", "SECAO_N"])["QT_VOTOS_N"]
                .sum().reset_index(name="VOTOS_CARGO"))
    comp = (sub.groupby(["ZONA_N", "SECAO_N"])["QT_COMPARECIMENTO"]
               .first().reset_index())
    comp["COMP_N"] = pd.to_numeric(comp["QT_COMPARECIMENTO"], errors="coerce")

    conf = votos.merge(comp[["ZONA_N", "SECAO_N", "COMP_N"]],
                       on=["ZONA_N", "SECAO_N"], how="left")
    conf["DIF"] = conf["VOTOS_CARGO"] - conf["COMP_N"]
    divergentes = conf[(conf["DIF"].notna()) & (conf["DIF"] != 0)]
```

```
    print(f"\n{cargo}: {len(conf)} seções | {len(divergentes)} divergentes")
    if len(divergentes):
        print(divergentes.head(5).to_string(index=False))
```

```
63676 linhas · 36 colunas

Colunas: DT_GERACAO, HH_GERACAO, ANO_ELEICAO, CD_TIPO_ELEICAO, NM_TIPO_ELEICAO, CD_PLEITO, DT_PLEITO, NR_TURNO, C
=== Total de votos por cargo (individuais + agregadas) ===
DS_CARGO_PERGUNTA  QT_VOTOS_N
       Presidente     2232823
       Governador     2229510

=== Top 10 seções individuais ===
ZONA_N SECAO_N DS_TIPO_VOTAVEL  QT_VOTOS_N
     3     505         Nominal         838
     3     504         Nominal         834
     3     387         Nominal         785
     3     386         Nominal         779
     3     286         Nominal         777
     3     278         Nominal         777
     3     344         Nominal         775
     3     503         Nominal         774
     3     285         Nominal         769
     3     287         Nominal         765

=== Agregadas por cargo ===
                  linhas  votos
DS_CARGO_PERGUNTA
Governador            814  58557
Presidente            795  58611

=== GABARITO 2: seções com inconsistência ===
0 seções com QT_APTOS ≠ QT_COMPARECIMENTO + QT_ABSTENCOES

=== GABARITO 3: seções agregadas ===
1609 linhas com DS_SECOES_AGREGADAS preenchido

=== GABARITO 4: total geral ===
Total de votos no arquivo: 4,462,333

=== Votos por tipo ===
DS_TIPO_VOTAVEL  linhas   votos
        Nominal   32460 4264428
           Nulo   16178  142093
         Branco   15038   55812

=== Conferência: votos por cargo vs comparecimento ===

Governador: 7910 seções | 0 divergentes

Presidente: 7910 seções | 0 divergentes
```

Célula 4 — Parte 2: as ferramentas são funções Python comuns

```
def _parse_int(valor, nome):
    """Converte para int com mensagem de erro clara."""
    try:
        return int(valor), None
    except (TypeError, ValueError):
        return None, {"error": f"{nome} '{valor}' inválido. Use um número inteiro."}


def _num(v, default=0):
    """Converte para número com fallback seguro (NaN → default)."""
    n = pd.to_numeric(v, errors="coerce")
    return default if pd.isna(n) else n

def total_por_tipo(tipo_voto=None, cargo=None):
    """Total de votos por tipo, opcionalmente filtrado por cargo."""
    if tipo_voto is not None and tipo_voto not in ("Nulo", "Branco", "Nominal"):
        return {"error": f"tipo_voto '{tipo_voto}' inválido. Use 'Nulo', 'Branco' ou 'Nominal'."}
    if cargo is not None and cargo not in ("Governador", "Presidente"):
        return {"error": f"cargo '{cargo}' inválido. Use 'Governador' ou 'Presidente'."}

    r = bu.copy()
    r["QT_VOTOS_N"] = pd.to_numeric(r["QT_VOTOS"], errors="coerce").fillna(0)

    if tipo_voto is not None:
        r = r[r.DS_TIPO_VOTAVEL == tipo_voto]
    if cargo is not None:
        r = r[r.DS_CARGO_PERGUNTA == cargo]
```

```
    if r.empty:
        return {"error": "nenhum voto encontrado com esses filtros"}

    return {
        "total_votos": int(r["QT_VOTOS_N"].sum()),
        "tipo_voto": tipo_voto,
        "cargo": cargo,
        "linhas": len(r),
    }

def total_sem_biometria(zona=None, municipio=None, **kwargs):
    """Total de eleitores sem biometria válida, opcionalmente por zona ou município."""
    r = bu.copy()

    r["ZONA_N"] = pd.to_numeric(r["ZONA_N"], errors="coerce")
    r["SECAO_N"] = pd.to_numeric(r["SECAO_N"], errors="coerce")

    if zona is not None:
        z, err = _parse_int(zona, "zona")
        if err: return err
        if z == 0:
            return {"error": "zona 0 não existe. Omita o parâmetro para todas as zonas."}
        r = r[r["ZONA_N"] == z]

    if municipio is not None:
        r = r[r.NM_MUNICIPIO.str.upper() == str(municipio).upper()]

    if r.empty:
        return {"error": "nenhuma seção encontrada com esses filtros"}

    por_secao = r.drop_duplicates(subset=["ZONA_N", "SECAO_N"])

    biom = pd.to_numeric(por_secao["QT_ELEI_BIOM_SEM_HABILITACAO"],
                         errors="coerce").fillna(0)
    aptos_s = pd.to_numeric(por_secao["QT_APTOS"],
                            errors="coerce").fillna(0)

    total = int(biom.sum())
    aptos = int(aptos_s.sum())

    return {
        "total_sem_biometria": total,
        "total_aptos": aptos,
        "percentual": round(100 * total / aptos, 2) if aptos else None,
        "secoes": len(por_secao),
        "filtros": {"zona": zona, "municipio": municipio},
    }


def secao_details(zona, secao):
    """Todos os dados de uma seção."""
    z, err = _parse_int(zona, "zona")
    if err: return err
    s, err = _parse_int(secao, "secao")
    if err: return err

    r = bu.copy()
    r["ZONA_N"] = pd.to_numeric(r["ZONA_N"], errors="coerce")
    r["SECAO_N"] = pd.to_numeric(r["SECAO_N"], errors="coerce")
    r["QT_VOTOS_N"] = pd.to_numeric(r["QT_VOTOS"], errors="coerce").fillna(0)

    r = r[(r["ZONA_N"] == z) & (r["SECAO_N"] == s)]

    if r.empty:
        return {"error": f"seção {z}/{s} não encontrada"}

    primeira = r.iloc[0]

    votos = (r.groupby(["DS_CARGO_PERGUNTA", "DS_TIPO_VOTAVEL"], as_index=False)["QT_VOTOS_N"]
               .sum()
               .assign(QT_VOTOS=lambda d: d["QT_VOTOS_N"].astype(int))
               .drop(columns="QT_VOTOS_N")
               .to_dict(orient="records"))

    return {
        "zona": int(z),
        "secao": int(s),
        "municipio": primeira["NM_MUNICIPIO"],
        "aptos": int(_num(primeira["QT_APTOS"])),
        "comparecimento": int(_num(primeira["QT_COMPARECIMENTO"])),
        "abstencoes": int(_num(primeira["QT_ABSTENCOES"])),
        "sem_biometria": int(_num(primeira["QT_ELEI_BIOM_SEM_HABILITACAO"])),
```

```
            "votos": votos,
        }


    def compare_secoes(zona_a, secao_a, zona_b, secao_b, tipo_voto=None):
        """Compara duas seções lado a lado, agregando por cargo."""
        if tipo_voto is not None and tipo_voto not in ("Nulo", "Branco", "Nominal"):
            return {"error": f"tipo_voto '{tipo_voto}' inválido."}

        pares = [("zona_a", zona_a), ("secao_a", secao_a),
                 ("zona_b", zona_b), ("secao_b", secao_b)]
        ids = []
        for nome, valor in pares:
            v, err = _parse_int(valor, nome)
            if err: return err
            ids.append(v)

        base = bu.copy()
        base["ZONA_N"] = pd.to_numeric(base["ZONA_N"], errors="coerce")
        base["SECAO_N"] = pd.to_numeric(base["SECAO_N"], errors="coerce")
        base["QT_VOTOS_N"] = pd.to_numeric(base["QT_VOTOS"], errors="coerce").fillna(0)

        secoes = []
        for i in range(0, 4, 2):
            z, s = ids[i], ids[i + 1]
            r_full = base[(base["ZONA_N"] == z) & (base["SECAO_N"] == s)]
            if r_full.empty:
                return {"error": f"seção {z}/{s} não encontrada"}

            r = r_full if tipo_voto is None else r_full[r_full.DS_TIPO_VOTAVEL == tipo_voto]

            primeira = r_full.iloc[0]
            votos_cargo = (r.groupby("DS_CARGO_PERGUNTA")["QT_VOTOS_N"]
                           .sum().to_dict())
            secoes.append({
                "chave": f"{z}/{s}",
                "NR_ZONA": int(primeira["ZONA_N"]),
                "NR_SECAO": int(primeira["SECAO_N"]),
                "NM_MUNICIPIO": primeira["NM_MUNICIPIO"],
                "QT_APTOS": int(_num(primeira["QT_APTOS"])),
                "QT_COMPARECIMENTO": int(_num(primeira["QT_COMPARECIMENTO"])),
                "QT_ABSTENCOES": int(_num(primeira["QT_ABSTENCOES"])),
                "votos_por_cargo": {k: int(v) for k, v in votos_cargo.items()},
                "total_votos": int(r["QT_VOTOS_N"].sum()),
                "AGREGADA": bool(primeira["AGREGADA"]),
                "DS_SECOES_AGREGADAS": primeira["DS_SECOES_AGREGADAS"],
            })

        a, b = secoes[0], secoes[1]
        best = {
            "mais_aptos": [a["chave"] if a["QT_APTOS"] >= b["QT_APTOS"] else b["chave"]],
            "mais_comparecimento": [a["chave"] if a["QT_COMPARECIMENTO"] >= b["QT_COMPARECIMENTO"] else b["chave"]],
            "mais_abstencoes": [a["chave"] if a["QT_ABSTENCOES"] >= b["QT_ABSTENCOES"] else b["chave"]],
            "mais_votos": [a["chave"] if a["total_votos"] >= b["total_votos"] else b["chave"]],
        }

        vencedor = best["mais_votos"][0]
        resposta = (f"{a['chave']} tem mais votos: {a['total_votos']} vs {b['total_votos']}"
                    if vencedor == a["chave"]
                    else f"{b['chave']} tem mais votos: {b['total_votos']} vs {a['total_votos']}")

        resultado = {
            "resposta": resposta,
            "secoes": secoes,
            "best": best,
            "tipo_voto_aplicado": tipo_voto,
        }

        agregadas = [s["chave"] for s in secoes if s["AGREGADA"]]
        if agregadas:
            resultado["aviso"] = (
                f"As seções {', '.join(agregadas)} são CONSOLIDADAS. "
                f"QT_APTOS, QT_COMPARECIMENTO e QT_ABSTENCOES não são "
                f"comparáveis entre elas e uma seção individual."
            )

        return resultado


    def search_secoes(tipo_voto=None, zona=None, cargo=None, min_votos=None, top_n=5):
        """Busca seções no BU por tipo de voto, zona, cargo e mínimo de votos."""
```

```
    TIPOS = ("Nulo", "Branco", "Nominal")
    CARGOS = ("Governador", "Presidente")

    # tolerância: se cargo recebeu um tipo de voto, move para tipo_voto
    if cargo in TIPOS:
        if tipo_voto is None:
            tipo_voto = cargo
        cargo = None

    if cargo is not None and cargo not in CARGOS:
        return {"error": f"cargo '{cargo}' inválido. Use 'Governador' ou 'Presidente'."}
    if tipo_voto is not None and tipo_voto not in TIPOS:
        return {"error": f"tipo_voto '{tipo_voto}' inválido. Use 'Nulo', 'Branco' ou 'Nominal'."}

    r = bu.copy()
    r["ZONA_N"] = pd.to_numeric(r["ZONA_N"], errors="coerce")
    r["SECAO_N"] = pd.to_numeric(r["SECAO_N"], errors="coerce")
    r["QT_VOTOS_N"] = pd.to_numeric(r["QT_VOTOS"], errors="coerce").fillna(0)

    if tipo_voto is not None:
        r = r[r.DS_TIPO_VOTAVEL == tipo_voto]

    if zona is not None:
        zona_n, err = _parse_int(zona, "zona")
        if err: return err
        if zona_n == 0:
            return {"error": "zona 0 não existe. Omita o parâmetro zona para buscar em todas as zonas."}
        r = r[r["ZONA_N"] == zona_n]

    if cargo is not None:
        r = r[r.DS_CARGO_PERGUNTA == cargo]

    if min_votos is not None:
        mv, err = _parse_int(min_votos, "min_votos")
        if err: return err
        r = r[r["QT_VOTOS_N"] >= mv]

    if r.empty:
        return {"error": "nenhuma seção encontrada com esses filtros"}

    agg = (r.groupby(["ZONA_N", "SECAO_N", "NM_MUNICIPIO"], as_index=False)["QT_VOTOS_N"]
             .sum()
             .sort_values("QT_VOTOS_N", ascending=False))

    top = agg.head(int(top_n or 5))

    return {
        "total_found": len(agg),
        "resumo": f"{len(agg)} seções atendem ao filtro. Abaixo, as {len(top)} com mais votos.",
        "filtros_aplicados": {"tipo_voto": tipo_voto, "zona": zona,
                              "cargo": cargo, "min_votos": min_votos},
        "secoes": top.to_dict(orient="records"),
    }
```

Célula 5 — Parte 3: a descrição das ferramentas

📋 Parte 3b · TOOLS completo (schemas limpos)

```
# @title 📋 Parte 3b · TOOLS completo (schemas limpos)

TOOLS = [
    {"type": "function",
     "function": {
         "name": "search_secoes",
         "description": ("Busca seções no Boletim de Urna por tipo de voto "
                         "(Nulo, Branco, Nominal), zona eleitoral, cargo "
                         "(Governador ou Presidente) e mínimo de votos. "
                         "Devolve quantas seções bateram no filtro e as 5 com "
                         "mais votos."),
         "parameters": {
             "type": "object",
             "properties": {
                 "tipo_voto": {
                     "type": "string",
                     "enum": ["Nulo", "Branco", "Nominal"],
                     "description": ("Tipo de voto a filtrar. OMITA para buscar "
                                     "todos os tipos.")
                 },
                 "zona": {
```

```
                    "type": "integer",
                    "description": ("Número da zona eleitoral. OMITA para buscar "
                                    "em todas as zonas. NÃO passe 0.")
                },
                "cargo": {
                    "type": "string",
                    "enum": ["Governador", "Presidente"],
                    "description": ("Cargo a filtrar. NÃO use 'Nulo', 'Branco' "
                                    "ou 'Nominal' aqui. OMITA para todos os cargos.")
                },
                "min_votos": {
                    "type": "integer",
                    "description": "Mínimo de votos na seção"
                },
                "top_n": {
                    "type": "integer",
                    "description": "Quantas seções devolver (padrão 5)"
                },
            },
            "required": [],
        }}},
    {"type": "function",
     "function": {
        "name": "total_sem_biometria",
        "description": ("Total de eleitores sem biometria válida no arquivo, "
                        "opcionalmente filtrado por zona ou município. "
                        "Devolve também o percentual sobre os aptos. "
                        "Aceita APENAS zona e municipio — NÃO aceita tipo_voto, "
                        "cargo, min_votos nem top_n."),
        "parameters": {
            "type": "object",
            "properties": {
                "zona": {
                    "type": "integer",
                    "description": "Número da zona. OMITA para todas as zonas."
                },
                "municipio": {
                    "type": "string",
                    "description": "Nome do município. OMITA para todos."
                },
            },
            "required": [],
       }}},

    {"type": "function",
     "function": {
        "name": "secao_details",
        "description": ("Todos os dados de uma seção: aptos, comparecimento, "
                        "abstenções, horários, eleitores sem biometria, e os "
                        "votos agregados por cargo e tipo de voto."),
        "parameters": {
            "type": "object",
            "properties": {
                "zona": {"type": "integer", "description": "Número da zona"},
                "secao": {"type": "integer", "description": "Número da seção"},
            },
            "required": ["zona", "secao"],
        }}},

    {"type": "function",
     "function": {
        "name": "compare_secoes",
        "description": ("Compara duas seções lado a lado (aptos, "
                        "comparecimento, abstenções, votos por cargo). "
                        "Devolve o total de votos de cada seção e o vencedor "
                        "de cada critério."),
        "parameters": {
            "type": "object",
            "properties": {
                "zona_a": {"type": "integer"},
                "secao_a": {"type": "integer"},
                "zona_b": {"type": "integer"},
                "secao_b": {"type": "integer"},
                "tipo_voto": {
                    "type": "string",
                    "enum": ["Nulo", "Branco", "Nominal"],
                    "description": ("Tipo de voto a comparar. OMITA para somar "
                                    "todos os tipos.")
                },
            },
            "required": ["zona_a", "secao_a", "zona_b", "secao_b"],
        }}},
```

```
    {"type": "function",
      "function": {
          "name": "total_por_tipo",
          "description": ("Total de votos no arquivo inteiro, opcionalmente "
                          "filtrado por tipo de voto e/ou cargo. Use quando o "
                          "usuário pedir um total geral, não uma lista de seções."),
          "parameters": {
              "type": "object",
              "properties": {
                  "tipo_voto": {
                      "type": "string",
                      "enum": ["Nulo", "Branco", "Nominal"],
                      "description": "OMITA para somar todos os tipos de voto."
                  },
                  "cargo": {
                      "type": "string",
                      "enum": ["Governador", "Presidente"],
                      "description": "OMITA para somar os dois cargos."
                  },
              },
              "required": [],
          }}},
]

FUNCTIONS = {
    "search_secoes": search_secoes,
    "secao_details": secao_details,
    "compare_secoes": compare_secoes,
    "total_por_tipo": total_por_tipo,
    "total_sem_biometria": total_sem_biometria,
}
```

Célula 6 — Parte 4: o system prompt

```
SYSTEM = """Agente de auditoria do BU das Eleições 2026 do RN.

FERRAMENTAS:
- search_secoes: busca seções e devolve o TOP N por votos. Parâmetros: tipo_voto, cargo, zona, min_votos, top_n.
- secao_details: dados completos de uma seção (aptos, comparecimento, abstenções, biometria, votos por cargo/tip
- compare_secoes: compara duas seções e identifica qual tem mais aptos, comparecimento, abstenções e votos.
- total_por_tipo: TOTAL de votos no arquivo. Parâmetros: tipo_voto, cargo.
- total_sem_biometria: total de eleitores sem biometria válida. Parâmetros: zona, municipio.

ROTEAMENTO:
- "quantos", "total", "somando", "no total" → total_por_tipo
- "quais seções", "onde", "top", "mais votos" → search_secoes
- "detalhes da seção X" → secao_details
- "compara X com Y" → compare_secoes
- "sem biometria" → total_sem_biometria
- NUNCA use search_secoes para responder "quantos" — ela devolve o top por seção, não o total.

PARÂMETROS:
- cargo aceita APENAS "Governador" ou "Presidente".
- tipo_voto aceita APENAS "Nulo", "Branco" ou "Nominal".
- NUNCA passe "Nulo", "Branco" ou "Nominal" em cargo.

REGRAS:
1. Se o usuário não citou um filtro, NÃO passe esse parâmetro.
   "no total" ou "no arquivo" significa sem filtro.
2. Se a ferramenta devolver "error", corrija os parâmetros e chame
   de novo. NÃO responda ao usuário com o erro.
   Se o erro for de parâmetro inválido, leia a mensagem e ajuste:
   "Nulo", "Branco" e "Nominal" são tipo_voto, nunca cargo.
   Se o erro disser que um parâmetro não existe, remova-o e chame de novo.
3. Quando a ferramenta devolver dados, responda com os dados e
   ENCERRE. Não acrescente frases de encerramento, ressalvas ou
   avisos ao final.
4. FORA DO ESCOPO: o agente só responde sobre o BU (votos por seção,
   zona, cargo e tipo). Perguntas sobre pesquisas eleitorais, nomes
   de candidatos (Lula, Bolsonaro, qualquer político), geografia,
   história, conhecimento geral — NÃO responda com conhecimento
   próprio. Diga apenas que o dado não está no arquivo e pare.
   Exemplo: "Qual é a capital do RN?" → "Esse dado não está no arquivo."
5. Em compare_secoes, use o campo "resposta" como está, citando os
   DOIS números. NÃO recalcule.
6. Nunca invente números. Cite zona e seção.
7. Ao citar números, copie exatamente como aparecem na observação da ferramenta.
   Nunca reformate: 4.264.428 não é 426.4428. 55.812 não é 55,812.
```

```
8. Ao listar seções, escreva "Zona X, Seção Y" — nunca os nomes de coluna ZONA_N ou SECAO_N.
9. Ao comparar seções, cite SEMPRE os dois totais, no formato "X tem mais: A vs B".
   Nunca omita o número da seção perdedora.
10. Ao citar percentuais, copie exatamente: 6,8% — nunca 6.8%.
11. Ao citar municípios, preserve o nome exatamente como aparece na observação.

Responda em português, direto."""

PEDIDO = "Quais seções têm mais votos nulos para presidente?"
print("system prompt definido")
```

```
system prompt definido
```

Célula 7 — Parte 5: o loop completo

🔁 Parte 5b · summary completo (todas as ferramentas)

```
# @title 🔁 Parte 5b · summary completo (todas as ferramentas)

def summary(result):
    if not isinstance(result, dict):
        return str(result)

    if "error" in result:
        return f'❌ erro: {result["error"]}'

    if "total_found" in result:
        secoes = [f'{s["ZONA_N"]}/{s["SECAO_N"]} ({s["NM_MUNICIPIO"]}): {s["QT_VOTOS_N"]}'
                  for s in result.get("secoes", [])]
        return f'{result["total_found"]} seções. Top: ' + " | ".join(secoes)

    if "total_sem_biometria" in result:
        pct = result.get("percentual")
        pct_txt = f" ({str(pct).replace('.', ',')}% dos aptos)" if pct is not None else ""
        total = f'{result["total_sem_biometria"]:,}'.replace(",", ".")
        return f'{total} sem biometria{pct_txt} em {result["secoes"]} seções'

    if "total_votos" in result and "secoes" not in result:
        filtro = result.get("tipo_voto") or "todos os tipos"
        cargo = result.get("cargo")
        if cargo:
            filtro = f"{filtro} para {cargo}"
        linhas = result.get("linhas")
        total = f'{result["total_votos"]:,}'.replace(",", ".")
        base = f'{total} votos ({filtro})'
        return f'{base} em {linhas} linhas' if linhas is not None else base

    if "resposta" in result:
        return result["resposta"]

    if "votos" in result:
        por_cargo = {}
        for v in result["votos"]:
            por_cargo.setdefault(v["DS_CARGO_PERGUNTA"], 0)
            por_cargo[v["DS_CARGO_PERGUNTA"]] += v["QT_VOTOS"]
        totais = " | ".join(f'{k}: {v}' for k, v in por_cargo.items())
        return (f'Seção {result["zona"]}/{result["secao"]} — {result["municipio"]}: '
                f'{result["aptos"]} aptos, {result["comparecimento"]} comparecimento. '
                f'Votos por cargo: {totais}')

    return str(result)[:300]

def run_agent(pedido, max_steps=5):
    messages = [{"role": "system", "content": SYSTEM},
                {"role": "user", "content": pedido}]
    print(f"👤 {pedido}\n")          # ← única impressão da pergunta
    for volta in range(1, max_steps + 1):
        print(f"── volta {volta} " + "─" * 60)
        msg = llm(messages, tools=TOOLS)["message"]
        messages.append(msg)
        if not msg.get("tool_calls"):
            print("🤖 o modelo respondeu com texto → fim do loop\n")
            print(msg["content"])     # ← única impressão da resposta
            return messages
        for c in msg["tool_calls"]:
            name = c["function"]["name"]
            raw = c["function"]["arguments"]
            args = raw if isinstance(raw, dict) else json.loads(raw or "{}")
            print(f"🤖 o modelo pediu     → {name}({json.dumps(args, ensure_ascii=False)})")
            try:
```

```
            try:
                result = FUNCTIONS[name](**args)
            except Exception as e:
                result = {"error": f"{type(e).__name__}: {e}"}
            print(f"🔧 nós executamos    → {summary(result)}")
            messages.append({"role": "tool", "tool_name": name,
                             "content": json.dumps(result, ensure_ascii=False)})
    print(f"(parei depois de {max_steps} voltas sem resposta final)")
    return messages


conversa = run_agent(PEDIDO)
```

```
👤 Quais seções têm mais votos nulos para presidente?

── volta 1 ──────────────────────────────────────────────────────────
🤖 o modelo pediu     → search_secoes({"tipo_voto": "Nulo", "cargo": "Presidente"})
🔧 nós executamos    → 8083 seções. Top: 25/105 (CAICÓ): 26 | 4/16 (NATAL): 23 | 51/176 (SÃO GONÇALO DO AMARANTE)
── volta 2 ──────────────────────────────────────────────────────────
🤖 o modelo respondeu com texto → fim do loop

As seções com mais votos nulos para presidente são:

- Zona 25, Seção 105, município CAICÓ, com 26 votos.
- Zona 4, Seção 16, município NATAL, com 23 votos.
- Zona 51, Seção 176, município SÃO GONÇALO DO AMARANTE, com 22 votos.
- Zona 69, Seção 281, município NATAL, com 22 votos.
- Zona 1, Seção 352, município NATAL, com 22 votos.
```

🧪 Parte 6b · Bateria final

```
# @title 🧪 Parte 6b · Bateria final

TESTES = [
    {"pergunta": "Quais seções têm mais votos nulos para presidente?",
     "esperado": "Caicó, zona 25 seção 105, com 26 votos nulos",
     "tipo": "resposta", "checar": ["25", "105", "26"],
     "proibido": ["não existe", "não encontrad"]},

   {"pergunta": "Quais seções têm mais votos nulos para governador?",
    "esperado": "Parelhas, zona 24 seção 35, com 37 votos nulos",
    "tipo": "resposta",
    "checar": ["24", "35", "37"],
    "proibido": ["não existe", "não encontrada"]},

    {"pergunta": "Me dá os detalhes da seção 505 da zona 3.",
     "esperado": "Seção existe, com votos de Governador e Presidente",
     "tipo": "resposta", "checar": ["505", "governador", "presidente"],
     "proibido": ["não existe", "não encontrad"]},

    {"pergunta": "Compara os votos nominais da seção 505 com a 504 da zona 3. Qual tem mais?",
     "esperado": "505 tem mais (838 vs 834 nominais)",
     "tipo": "resposta",
     "checar": ["505", "838"],
     "proibido": ["não existe", "não encontrada", "504 tem mais"]},

    {"pergunta": "Qual o total de votos nominais no arquivo?",
     "esperado": "4.264.428",
     "tipo": "resposta", "checar": ["4.264.428", "4264428"],
     "proibido": ["não existe"]},

    {"pergunta": "Quantos votos brancos existem no total?",
     "esperado": "55.812",
     "tipo": "resposta", "checar": ["55.812", "55812"],
     "proibido": ["não existe"]},

    {"pergunta": "Qual foi a intenção de voto para governador nas pesquisas?",
     "esperado": "RECUSA — dado não existe no arquivo",
     "tipo": "recusa",
     "checar": ["não existe", "não tenho", "não posso", "não fornece"]},

    {"pergunta": "Quantos votos o candidato Lula teve?",
     "esperado": "RECUSA — dado não existe no arquivo",
     "tipo": "recusa",
     "checar": ["não existe", "não tenho", "não posso"]},

    {"pergunta": "Qual é a capital do Rio Grande do Norte?",
     "esperado": "RECUSA — conhecimento externo, fora do escopo",
     "tipo": "recusa",
     "checar": ["não existe", "não posso", "fora do escopo",
                "só respondo", "apenas sobre"]},
    {"pergunta": "Quantos eleitores estão sem biometria válida no total?",
```

```
     "esperado": "180.838 sem biometria (6,8% dos aptos)",
     "tipo": "resposta",
     "checar": ["180.838", "6,8"],
     "proibido": ["não existe", "não está"]},

    {"pergunta": "Quais seções têm mais votos nulos?",
     "esperado": "Carnaúba dos Dantas, zona 22 seção 41, com 52 votos nulos",
     "tipo": "resposta",
     "checar": ["22", "41", "52"],
     "proibido": ["não existe", "não encontrada"]},
]


def extrair_resposta(conversa):
    """Pega a última mensagem de texto do assistente."""
    for m in reversed(conversa or []):
        if m.get("role") == "assistant" and m.get("content"):
            return m["content"]
    return ""


def avaliar(teste, resposta):
    texto = (resposta or "").lower()
    texto_num = texto.replace(".", "").replace(",", "")

    for p in teste.get("proibido", []):
        if p.lower() in texto:
            return False, [f"contém proibido: '{p}'"]

    if teste["tipo"] == "resposta":
        faltando = []
        for c in teste["checar"]:
            c_norm = c.lower().replace(".", "").replace(",", "")
            if c.lower() not in texto and c_norm not in texto_num:
                faltando.append(c)
        return len(faltando) == 0, faltando

    # recusa: aceita variações naturais
    gatilhos = [
        "não existe", "não está", "não tenho", "não posso",
        "não fornece", "não disponível", "não consta",
        "fora do escopo", "não faz parte", "não há",
        "só respondo", "apenas sobre", "não contempla",
    ]
    achou = any(g in texto for g in gatilhos)
    return achou, [] if achou else teste["checar"]


resultados = []
for i, t in enumerate(TESTES, 1):
    print(f"\n{'=' * 72}")
    print(f"TESTE {i} [{t['tipo'].upper()}]")
    print(f"Esperado: {t['esperado']}")
    print(f"{'=' * 72}")

    conversa = run_agent(t["pergunta"])
    resposta = extrair_resposta(conversa)
    ok, faltando = avaliar(t, resposta)
    resultados.append({"teste": i, "ok": ok, "faltando": faltando,
                       "resposta": resposta[:200]})
    print(f"\n{'✅ PASSOU' if ok else '❌ FALHOU'}"
          + (f" — {faltando}" if faltando else ""))

passou = sum(1 for r in resultados if r["ok"])
print(f"\n{'=' * 72}")
print(f"PLACAR: {passou}/{len(TESTES)} testes passaram")
print(f"{'=' * 72}")
for r in resultados:
    print(f"  Teste {r['teste']}: {'✅' if r['ok'] else '❌'}"
          + (f" — {r['faltando']}" if r['faltando'] else ""))
```

```
🤖 o modelo pediu     → total_sem_biometria({"zona": null, "municipio": null})
🔧 nós executamos     → 180.838 sem biometria (6,8% dos aptos) em 8115 seções
── volta 2 ──────────────────────────────────────────────────────────
🤖 o modelo respondeu com texto → fim do loop

No total, há 180.838 eleitores sem biometria válida, correspondendo a 6,8% dos aptos. Esses dados são de 8115 s

✅ PASSOU

══════════════════════════════════════════════════════════════════
TESTE 11 [RESPOSTA]
Esperado: Carnaúba dos Dantas, zona 22 seção 41, com 52 votos nulos
══════════════════════════════════════════════════════════════════
👤 Quais seções têm mais votos nulos?

── volta 1 ──────────────────────────────────────────────────────────
🤖 o modelo pediu     → search_secoes({"tipo_voto": "Nulo", "top_n": 5})
🔧 nós executamos     → 8109 seções. Top: 22/41 (CARNAÚBA DOS DANTAS): 52 | 1/352 (NATAL): 51 | 24/35 (PARELHAS)
── volta 2 ──────────────────────────────────────────────────────────
🤖 o modelo respondeu com texto → fim do loop

As seções com mais votos nulos são:

- Zona 22, Seção 41, município CARNAÚBA DOS DANTAS, com 52 votos.
- Zona 1, Seção 352, município NATAL, com 51 votos.
- Zona 24, Seção 35, município PARELHAS, com 50 votos.
- Zona 69, Seção 106, município NATAL, com 49 votos.
- Zona 20, Seção 59, município CURRAIS NOVOS, com 49 votos.

✅ PASSOU

══════════════════════════════════════════════════════════════════
PLACAR: 11/11 testes passaram
══════════════════════════════════════════════════════════════════
  Teste 1: ✅
  Teste 2: ✅
  Teste 3: ✅
  Teste 4: ✅
  Teste 5: ✅
  Teste 6: ✅
  Teste 7: ✅
  Teste 8: ✅
  Teste 9: ✅
  Teste 10: ✅
  Teste 11: ✅
```

💬 Perguntas livres em loop

```
# @title 💬 Perguntas livres em loop

print("Digite 'sair' para encerrar.\n")

while True:
    pergunta = input("👤 ")
    if pergunta.strip().lower() in ("sair", "exit", "q"):
        break
    if not pergunta.strip():
        continue

    run_agent(pergunta)      # ← sem print aqui
```

```
👤 quantos votos nulos houveram?
👤 quantos votos nulos houveram?

── volta 1 ──────────────────────────────────────────────────────────
🤖 o modelo pediu     → total_por_tipo({"cargo": null, "tipo_voto": "Nulo"})
🔧 nós executamos     → 142.093 votos (Nulo) em 16178 linhas
── volta 2 ──────────────────────────────────────────────────────────
🤖 o modelo respondeu com texto → fim do loop

No total, houveram 142.093 votos nulos.
👤 sair
```

```
run_agent("quantos votos sem biometria?")
```

```
👤 quantos votos sem biometria?

── volta 1 ──────────────────────────────────────────────────────────
🤖 o modelo pediu     → total_sem_biometria({"zona": null})
🔧 nós executamos     → 180.838 sem biometria (6,8% dos aptos) em 8115 seções
── volta 2 ──────────────────────────────────────────────────────────
🤖 o modelo respondeu com texto → fim do loop

No total, há 180.838 eleitores sem biometria válida, representando 6,8% do total de aptos, que é de 2.659.148 ele
[{'role': 'system',
  'content': 'Agente de auditoria do BU das Eleições 2026 do RN.\n\nFERRAMENTAS:\n- search_secoes: busca seções
e devolve o TOP N por votos. Parâmetros: tipo_voto, cargo, zona, min_votos, top_n.\n- secao_details: dados
```

```
completos de uma seção (aptos, comparecimento, abstenções, biometria, votos por cargo/tipo).\n- compare_secoes:
compara duas seções e identifica qual tem mais aptos, comparecimento, abstenções e votos.\n- total_por_tipo:
TOTAL de votos no arquivo. Parâmetros: tipo_voto, cargo.\n- total_sem_biometria: total de eleitores sem
biometria válida. Parâmetros: zona, municipio.\n\nROTEAMENTO:\n- "quantos", "total", "somando", "no total" →
total_por_tipo\n- "quais seções", "onde", "top", "mais votos" → search_secoes\n- "detalhes da seção X" →
secao_details\n- "compara X com Y" → compare_secoes\n- "sem biometria" → total_sem_biometria\n- NUNCA use
search_secoes para responder "quantos" — ela devolve o top por seção, não o total.\n\nPARÂMETROS:\n- cargo
aceita APENAS "Governador" ou "Presidente".\n- tipo_voto aceita APENAS "Nulo", "Branco" ou "Nominal".\n- NUNCA
passe "Nulo", "Branco" ou "Nominal" em cargo.\n\nREGRAS:\n1. Se o usuário não citou um filtro, NÃO passe esse
parâmetro.\n   "no total" ou "no arquivo" significa sem filtro.\n2. Se a ferramenta devolver "error", corrija os
parâmetros e chame\n   de novo. NÃO responda ao usuário com o erro.\n   Se o erro for de parâmetro inválido,
leia a mensagem e ajuste:\n   "Nulo", "Branco" e "Nominal" são tipo_voto, nunca cargo.\n   Se o erro disser que
um parâmetro não existe, remova-o e chame de novo.\n3. Quando a ferramenta devolver dados, responda com os dados
e\n   ENCERRE. Não acrescente frases de encerramento, ressalvas ou\n   avisos ao final.\n4. FORA DO ESCOPO: o
agente só responde sobre o BU (votos por seção,\n   zona, cargo e tipo). Perguntas sobre pesquisas eleitorais,
nomes\n   de candidatos (Lula, Bolsonaro, qualquer político), geografia,\n   história, conhecimento geral — NÃO
responda com conhecimento\n   próprio. Diga apenas que o dado não está no arquivo e pare.\n   Exemplo: "Qual é a
capital do RN?" → "Esse dado não está no arquivo."\n5. Em compare_secoes, use o campo "resposta" como está,
citando os\n   DOIS números. NÃO recalcule.\n6. Nunca invente números. Cite zona e seção.\n7. Ao citar números,
copie exatamente como aparecem na observação da ferramenta.\n   Nunca reformate: 4.264.428 não é 426.4428.
55.812 não é 55,812.\n8. Ao listar seções, escreva "Zona X, Seção Y" — nunca os nomes de coluna ZONA_N ou
SECAO_N.\n9. Ao comparar seções, cite SEMPRE os dois totais, no formato "X tem mais: A vs B".\n   Nunca omita o
número da seção perdedora.\n10. Ao citar percentuais, copie exatamente: 6,8% — nunca 6.8%.\n11. Ao citar
municípios, preserve o nome exatamente como aparece na observação.\n\nResponda em português, direto.'},
 {'role': 'user', 'content': 'quantos votos sem biometria?'},
 {'role': 'assistant',
  'content': '',
  'tool_calls': [{'id': 'call_z64c93ph',
    'function': {'index': 0,
     'name': 'total_sem_biometria',
     'arguments': {'zona': None}}}]},
 {'role': 'tool',
  'tool_name': 'total_sem_biometria',
  'content': '{"total_sem_biometria": 180838, "total_aptos": 2659148, "percentual": 6.8, "secoes": 8115,
"filtros": {"zona": null, "municipio": null}}'},
 {'role': 'assistant',
  'content': 'No total, há 180.838 eleitores sem biometria válida, representando 6,8% do total de aptos, que é
de 2.659.148 eleitores. Esses dados são de 8.115 seções.'}]
```